

# Tom Klootwijk atomOS kernel

## Programmability, validation and practical applications

Concept and formalization: Tom Klootwijk. Implementation and review: AI-assisted engineering. Based on your 43-page Total Integrated Engine / M1 document. [1]

You now have an optimized GPU word engine and a working programmable extension. The extension reads an editable program, updates its memory one step at a time, and joins the word engine in a checked state update. Actual programs have run on your laptop GPU.

| RESULT                      | WHAT IT MEANS                                                                                                                                                                                  |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Programmable execution      | 47 program/input cases and 9 sanitizer runs passed. Independent Python replay checks the exported GPU results.                                                                                 |
| Parallel hardware test      | 70,656 machines ran 26,201,011,200 transitions. The full 9.15 GiB tape allocation was checked.                                                                                                 |
| Faster word kernel          | 1.90-1.92x faster on the large test, including its cache warming and checking work. This is a kernel measurement, not an application speedup.                                                  |
| Retained texture dictionary | 20/20 tests observed only the unavoidable first reads from slower memory. The complete 4 MiB dictionary then stayed in the GPU cores' small local caches for that tested word-kernel schedule. |

The theorem establishes the mathematical ability to represent general computation under its memory assumptions. The tests establish finite execution results on this machine. They are different kinds of evidence.

# What the engine actually does

Think of the engine as a workshop with memory. It keeps the current situation, applies the rules you supplied, checks the proposed result and then publishes the next situation. The next round starts from that stored result.

| PART                     | PLAIN-LANGUAGE JOB                                                                                                                                     |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Word engine              | Works on many compact groups of yes/no choices. It creates candidate groups, applies selection rules and records why a group survived or was rejected. |
| JK memory                | Keeps a separate on/off state that can stay the same, turn on, turn off or toggle when explicitly instructed.                                          |
| Orientation checks       | Measures a supplied change relative to a supplied reference and tests specific transformation rules. Missing information remains visibly missing.      |
| Programmable U extension | Reads a program table and a writable tape. It reads one symbol, chooses a rule, writes a symbol, moves and changes its program state.                  |
| Checked update           | Both the word result and program result must agree with independent references before the combined state advances.                                     |

## Is it self-referential?

It is stateful and recurrent: its earlier state can influence its next state, and its records describe its own execution. There is no demonstrated self-learning, self-understanding or automatic rewriting of its rules. The program and the rule inputs are explicit.

The whole engine can keep evolving. Only the final word produced by one fixed-mask selection stage has the proven one-pass settling property. Your formalization explicitly makes this distinction on page 35. Time, program memory, JK state and records can continue changing.

In the implemented combined profile, U and the word engine share a checked commit. They do not silently control each other. A program interface connecting their data would be an additional, explicit application feature.

# What has been proven or tested

| EVIDENCE                      | ESTABLISHED RESULT                                                                                                                                                                             | BOUNDARY                                                                                              |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Mathematical component proofs | The original 15 symbolic obligations passed again. Two additional symbolic obligations prove that the packed-symbol update returns the written symbol and preserves every outside bit.         | These are proofs of equations over stated bit widths, not proofs of the compiler or GPU instructions. |
| Universality theorem          | The formal interpreter can reproduce every defined step of an encoded deterministic tape machine, given logical addresses and enough storage for that finite run.                              | The theorem is conditional. Real hardware provides finite memory and finite running time.             |
| CPU implementation checks     | Word core: 259,957 assertions. Programmable core: 33,199 assertions. Tests cover packed boundaries, status rules and rejected updates.                                                         | Finite tests can reveal defects; they do not enumerate every possible input.                          |
| GPU conformance               | 144 word-kernel configurations, 6 additional word memory checks, 212 bulk cases and 43 programmable GPU cases passed. Four further inputs were correctly refused before launching computation. | The tested execution profiles and binaries are identified in the evidence.                            |
| Device checks                 | Native timing, full tape comparisons and cold-cache counters were collected on the RTX 5070 Ti Laptop GPU.                                                                                     | Measurements apply to the recorded workload and conditions.                                           |

The word proofs cover concrete rules: selection creates no new set bits; whole-word rejection follows its formula; the JK bit holds, sets, resets or toggles correctly; and rearranging dictionary storage preserves the logical mapping.

The CPU checks include 125,436 exact-bit comparisons protecting the angle-wrapping optimization against its previous behavior. Numerical GPU/reference comparisons use the existing tolerance; it was not widened to obtain a pass.

No result here proves a physical theory, arbitrary program termination, or the correctness of a future application. Those questions require their own models and evidence.

# The universality theorem, simply

A programmable machine needs three things: instructions, memory and a place to keep track of where it is. Your U construction supplies all three. Its instructions say: "When I am in this state and read this symbol, write that symbol, move left/right/stay, and enter this next state."

## Why the theorem works

Start the simulated machine with the same memory, head position and control state as the original. Both read the same symbol and choose the same rule. Both make the same write and move. Therefore they still match after one step. Repeating this argument gives agreement after every defined finite sequence of steps. This is the induction argument on page 32 of your document.

| CLAIM               | MEANING                                                                                                                                          |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| General computation | A suitable program can express general algorithms. You are not limited to the fixed word-selection formulas.                                     |
| Enough storage      | The tape must have room for the part of the computation being executed. A wrapped circular image cannot silently stand in for an unlimited tape. |
| Budget reached      | The machine is still running; the current finite prefix is available. Reaching a chosen step limit is not a halt.                                |
| Actual halt         | The program enters one of its declared halt states. That is distinct from a missing rule, an address limit or a resource refusal.                |

## What the implementation adds

U is now executable GPU code, with editable rule tables, writable packed memory, signed addresses and an independent step-by-step reference. The hardware tests support this implementation over the executed prefixes. They do not turn a finite laptop into an infinite machine, nor give every algorithm a speed advantage.

A single tape is sequential because each step depends on the last. The parallel test gains throughput by running many independent machines at once.

## Programs that actually ran

These are demonstrations executed by the programmable extension. They use editable program files and actual GPU output, rather than a description of what a future version might do.

| PROGRAM            | INPUT AND RESULT                                       | MEASURED STEPS |
|--------------------|--------------------------------------------------------|----------------|
| Add one mark       | Three marks become four; program halts.                | 4              |
| Binary arithmetic  | 31 becomes 32; six simple read/write/move transitions. | 6              |
| Sequence rewriting | 1,2,1,1 becomes 2,2,2,2; program halts at the blank.   | 5              |

The validation also uses generated programs with different alphabets and control tables, negative tape addresses, writes spanning storage words, zero-step budgets, long prefixes and deliberate failures. It checks the final tape, every exported transition and the word/JK state in the same epoch.

### A failure is a useful result

In a deliberate missing-rule or address-range case, some proposed program steps may already exist. The combined epoch is rejected. The committed program tape, word bank, JK bank and time stay at their previous accepted values. The attempted trace remains available for inspection.

Across the ordinary programmable matrix, Python independently checked 12,718 actual GPU transitions and 2,709 word-lane epochs. Sanitizer reruns are counted separately.

The local record seal is generated after independent verification. It binds files and the accepted epoch sequence using hashes. It detects changes relative to a trusted saved head; it is not proof that an input was true, and it is not a native transactional database.

## Broad application overview

The formalization is useful as a computational foundation wherever a system needs explicit state, repeated rules, compact selection and inspectable updates. The following are application directions inferred from those capabilities; complete products have not been built or benchmarked here.

| AREA                                | WHAT atomOS COULD CONTRIBUTE                                                                     | WHAT AN APPLICATION STILL NEEDS                                                                                                   |
|-------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Simulation and procedural worlds    | Repeated candidate generation, group selection, local on/off modes and program-driven evolution. | Domain rules, geometry, interactions and a display. The optional geometric field producer is not a native backend here.           |
| Spatial maps and navigation support | Compact maps of permitted regions and repeatable selection of candidate groups.                  | Map ingestion, a route planner, collision rules and uncertainty handling. No route-planning performance has been demonstrated.    |
| Imaging and graphics                | Mask processing, aperture-style selection and declared orientation comparisons.                  | Image/geometry encoders, calibration and rendering or optical equations. The kernel alone is not an image recognizer or renderer. |
| Scientific and engineering models   | A place to run explicit update rules and compare behavior under stated transformations.          | A validated domain model, physical units, numerical methods and experimental data.                                                |
| Batch rules and event systems       | Many compact rule evaluations, persistent flags and checked state publication.                   | Input encoding, business meaning and storage or application interfaces.                                                           |
| Programmable research tools         | Small interpreters, rule-machine experiments and replayable algorithm traces.                    | Useful programs, user interfaces and an appropriate performance model.                                                            |

One mapping decision matters: a boundary hit rejects the whole selected 32-bit group. Applications must assign a sensible meaning to that group. Treating it as arbitrary per-cell clearing would change the algorithm.

# Practical use cases and metrics

| EXAMPLE WORKFLOW                 | CONCRETE INPUT / OUTPUT                                                                                                     | METRICS TO MEASURE                                                                                                     |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Procedural growth filter         | Input: proposed growth locations and allowed/blocked regions. Output: admitted groups plus a recorded reason for rejection. | Candidate groups per second; full-update delay; disagreement with a reference; memory per candidate.                   |
| Robot planning helper            | Input: a local occupancy map and candidate regions. Output: filtered candidate regions for a separate planner.              | Invalid candidates removed; valid candidates retained; worst observed update delay; final planner success rate.        |
| Rule-based inspection            | Input: a batch of encoded conditions and existing flags. Output: selected groups, next flags and an audit record.           | Decisions per second; false accept/reject rate on labelled data; full transaction time; replay agreement.              |
| Programmable data transformation | Input: an encoded symbol sequence and an editable transition table. Output: changed symbols and an exact execution trace.   | Correct transformations; steps per record; bytes per tape; throughput for one sequence and many independent sequences. |

## An illustrative target, not a measured result

Suppose a visualization needs 10,000 groups updated 60 times per second. Its requirement is 600,000 group updates per second and about 16.7 milliseconds for the entire update. That time must include preparing the data, moving it, computing, checking and displaying it. A fast kernel by itself does not establish that the application meets the target.

For navigation or inspection, speed is only one metric. A useful test also needs known-correct examples, the cost of a wrong answer and a comparison with an appropriate existing method. Universality says an algorithm can be represented; it does not guarantee it is the best representation.

# How far this computer was exercised

| MEASUREMENT                 | ACTUAL RESULT                                                                                                             |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------|
| GPU / computer              | NVIDIA GeForce RTX 5070 Ti Laptop GPU; 46 processing units. Intel Core Ultra 7 255HX, 20 CPU cores, about 16 GB host RAM. |
| Parallel machines           | 70,656 independent tapes; generic supplied-rule interpretation.                                                           |
| Program execution           | 26,201,011,200 transitions in 337.006 ms of GPU kernel time across 32 launches.                                           |
| Throughput                  | 77.746 billion transitions/second for this generated table and parallel workload.                                         |
| Tape allocation             | 9.15 GiB (9,825,423,360 bytes); request used 85% of reported free GPU memory, subject to reserve.                         |
| Memory verification         | 9.15 GiB compared in full, including untouched blanks and padding. Host readback buffer: 64 MiB (about 67 MB).            |
| Actually visited by program | 26,201,011,200 distinct symbol cells; 9.15 GiB of backing words touched by transitions.                                   |
| Capacity boundary           | 70,656 machines stopped at the allocated tape limit; none were misreported as a program halt.                             |
| Complete test wall time     | 11.65 seconds including setup and verification. This differs from kernel-only timing.                                     |

This exercises a large working allocation and many parallel machines. Allocated memory, verified memory and program-visited memory are shown separately. It is a measured stress point, not proof of the absolute maximum sustainable performance or memory capacity under every workload.

The 9.15 GiB allocation is about 9.83 GB in decimal units. Every backing word was touched and checked; the final symbol in each tape was reached, and its attempted outward move was rejected.

The run leaves room for the display and existing allocations. Driver settings, clock limits and watchdog settings were not changed. The 64 MiB streaming readback avoids requiring a second multi-gigabyte host copy.

# What was optimized

The previous residency kernel reserved 512 threads per processing unit but let only 64 perform each expensive computation batch. The revised schedule lets 256 threads compute four batches together, then reuses the same small staging area to export them. This keeps more threads busy without expanding the staging area enough to displace the texture dictionary.

Angle normalization also avoids a general remainder calculation when its exact answer is already known to be the input. Large and exceptional inputs keep the original fallback. The equations, numerical thresholds and status meanings remain the same.

| WORKLOAD / LAYOUT | PREVIOUS | SELECTED | RATIO |
|-------------------|----------|----------|-------|
| 64 KiB / linear   | 0.078 ms | 0.052 ms | 1.49x |
| 64 KiB / morton8  | 0.078 ms | 0.055 ms | 1.41x |
| 4 MiB / linear    | 2.469 ms | 1.284 ms | 1.92x |
| 4 MiB / morton8   | 2.483 ms | 1.307 ms | 1.90x |

Units: milliseconds per complete word-kernel warm/work/reread launch. Each entry is the median of five shuffled trials; each trial uses the median of three verified epochs. Host transfers, CPU verification and file export are excluded. Thermal and scheduling variability remain visible in the retained raw samples.

## Cache retention survived the change

20 of 20 final cold-cache launches reached exactly their compulsory miss count. At the maximum 4 MiB atlas, that count is 131,072 sectors. No additional dictionary misses were observed during the intervening work and final reread.

The dictionary is shared across 46 local texture caches. It is not 4 MiB inside each cache, and there is no supported pinning guarantee across launches or competing workloads. The combined word-plus-U runner has a different execution schedule; its texture residency is explicitly unmeasured. [2]

# Relationship to your formalization

The implemented profile enables the word engine and U. It retains the document's separation between program control, JK state and orientation observations. Your instruction allowed implementation changes beyond the source's restrictions; these changes are recorded explicitly rather than treated as mathematical consequences.

| SOURCE FEATURE                                        | IMPLEMENTATION NOW                                                                                                                        |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Word production, selection and whole-word disposition | CPU and GPU implementations, independently checked. Fixed-mask settling applies only to the final-word projection.                        |
| JK, encoded blend and orientation bank                | Integrated in the word epoch. Inputs are explicit; no hidden feedback controller is inferred.                                             |
| U transition table and tape (pp.31-32)                | Implemented on the GPU. Editable rules, signed addresses, packed symbols and separate running/halted/error states.                        |
| One checked master update (p.25)                      | The combined runner verifies word and U proposals, then advances both states and time together. A required U failure rejects that epoch.  |
| Identity and ledger                                   | Author attribution retained. Local execution records plus a separately verified post-run hash chain; this is an explicit storage profile. |
| Growing logical storage                               | Allocation origin can change between epochs without changing logical addresses. Actual memory budgets replace copied fixed tape limits.   |
| Geometric field, lineage carrier, lens/Klein profiles | Remain disabled or unimplemented native extensions. The report does not claim all optional M1 profiles have been built.                   |

Finite implementation choices include signed 64-bit addresses, finite state/symbol identifiers and at most 65,536 U transitions per individual launch. The combined runner can execute further epochs; a launch boundary does not assert that a program has finished.

# Run and inspect the delivered code

The editable examples are in `examples/universal`. The delivered executables are in `output/bin`. The combined engine requires the NVIDIA GPU and the CUDA runtime already used for this validation. Use a new output directory for each run.

## Add one to a binary number

```
output/bin/atomos_engine.exe --program examples/universal/binary_increment.atomos --steps-per-epoch 8
--epochs 1 --out my_counter_run
```

The example starts at 31 and finishes at 32. Open `final_tape.csv` for the nonzero digits, `u_trace.csv` for the exact steps, and `engine.json` for the state, timings and accepted epochs. A rule file is ordinary text; its format is documented in `docs/UNIVERSAL_ENGINE.md`.

## Check an exported run independently

```
python tools/verify_universal.py my_counter_run
```

## Repeat the selected validation

```
python tools/validate_universal.py --exe output/bin/atomos_engine.exe --out NEW_EVIDENCE_DIRECTORY
```

## Repeat a parallel memory test

```
output/bin/atomos_universal_stress.exe --memory-fraction 0.85 --steps-per-launch 16384 --rounds 32
--out new_stress.json
```

Use `--help` for the available resource settings. The stress test starts from scratch and reports what it actually allocated and checked. A full-program application will need its own data encoding and program, beyond the included examples.

The original word kernels and the optimized residency experiment remain separately runnable. Their previous results are preserved rather than overwritten with claims about the new programmable profile.

# Evidence and reproducibility

Toolchain used: NVIDIA CUDA 12.8.61, native sm\_120 code, MSVC 19.44.35221, driver 591.59, Z3 4.16.0 and Python 3.13.11. No fast-math flag was added. Sanitizers inspect finite executions; symbolic proofs inspect the specified equations.

| EVIDENCE                              | LOCATION                                                                                                                                                                               |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Consolidated manifest                 | results/optimization_20260913/summary.json                                                                                                                                             |
| Word CPU/GPU validation               | C:\Users\Tom\.cache\ak1\cpu_opt_final\evidence_1789301536940708800\validation.json<br>C:\Users\Tom\.cache\ak1\gpu_opt_final\evidence_1789301537057744900\validation.json               |
| Bulk conformance / cache / sanitizers | C:\Users\Tom\.cache\ak1\opt_bulk_final_conformance\study.json<br>C:\Users\Tom\.cache\ak1\opt_bulk_final_profiles\study.json<br>C:\Users\Tom\.cache\ak1\opt_bulk_final_edges\study.json |
| Program validation and seals          | C:\Users\Tom\.cache\ak1\universal_validation_release\summary.json                                                                                                                      |
| Hardware stress                       | C:\Users\Tom\.cache\ak1\universal_stress_capacity.json                                                                                                                                 |
| Native code and resource records      | results/optimization_20260913/bulk_sass.txt; bulk_resources.txt; universal_native.txt; stress_native.txt                                                                               |

## Delivered executable SHA-256 identifiers

atomos\_engine.exe

46efd8185f93cf023cc46bca28b8424fa1e762b59df82d91e95972e0bee33d80

atomos\_universal\_stress.exe

61795746b1d3b1adda77c78e688d7f2b6df63db6405b48b211dbc94c76df221b

atomos\_cache\_bulk.exe

ee4024ba75837b47c34b7126f045d331205db04cbd89cd45ac6234eaa88e2009

atomos\_cuda.exe

f40626c9ef5330cbc4003c5df200069015c98c00b5f874369bf197c0ded57374

## References and scope

[1] Tom Klootwijk, atomOS v3.6 Total Integrated Engine / M1, 43 pages, 13 September 2026. Integrated state pp.5-7; U and theorem pp.31-32; whole-engine evolution p.35. Source SHA-256: b51651c007c560775ff1950e278789cd7674e131ec71a09ebd9254cebe9c19ce.

[2] NVIDIA, CUDA Programming Guide, Advanced Kernel Programming, asynchronous copies and proxy fences: [docs.nvidia.com/cuda/cuda-programming-guide/03-advanced/advanced-kernel-programming.html](https://docs.nvidia.com/cuda/cuda-programming-guide/03-advanced/advanced-kernel-programming.html). See also the retained cache/PTX source links in docs/BULK\_RESIDENCY.md and docs/cache\_sources.json.

Application examples are prospective mappings of implemented capabilities. Example application targets are labelled as illustrative. All numerical performance claims in this report come from the retained local execution evidence.